

Laboratorio 4 — Comunicando con el servidor

Profesor: Matías Toro

Ayudantes: Sebastián Bustos, Julián Ferreira, Sebastián González, Camilo Huerta, Benjamín Indo, Benjamín Madrid, Tomás Méndez, Daniel Ramírez, Tomás Ubilla

El objetivo de este laboratorio es poner en práctica los conceptos vistos en la clase de *Comunicándose con el servidor*.

Se hará de nuevo la página de threads y comentarios, pero usando React. Tendrán que programar el front de la página conectándose con el servidor del ramo, para obtener de acá la información de los threads.

A modo de recordatorio, estas son las indicaciones del laboratorio con respecto a la aplicación:

a) **Listado de Threads:**

- La página principal mostrará un listado de threads.
- Un thread es un comentario creado desde la página principal.

b) **Comentarios:**

- Cada comentario debe estar asociado a un thread específico.
- Un comentario puede tener, de forma opcional, una referencia a otro comentario, lo que permite responder a un comentario en particular.
- Los atributos de un comentario son:
 - un identificador (id; pueden usar el id que asigna automáticamente mongo)
 - **Referencia al Thread:** Identificador del thread al que pertenece.
 - **Referencia opcional al Comentario Padre:** Identificador del comentario al que responde, en caso de existir.
 - **Fecha:** Momento en el que se realizó el comentario.
 - **Autor opcional:** Nombre del usuario que realiza el comentario. Si no hay usuario, se guarda como "Anonymous"
 - **Contenido:** Texto del comentario.

Requerimientos Técnicos:

• **Frontend:**

- La página principal debe mostrar el listado de threads con su contenido y autor.
- Al seleccionar un thread, se debe mostrar una vista detallada de un thread. En la vista detallada de un thread se incluye el thread seleccionado y su listado de comentarios.
- **Los comentarios deben obtenerse de forma dinámica, a partir de una base de datos.**

Requerimientos adicionales:

- Debe añadir funcionalidad de likes y dislikes para threads y comentarios. **No debe preocuparse de que la misma persona pueda dar like o dislike más de una vez.**
 - Al darle like o dislike a un comentario, la cantidad debe actualizarse sin necesitar recargar la página.
- En la vista detallada de un thread, se debe incluir un botón debajo de cada comentario para responder a ese comentario. El formulario para responder solo debe pedir contenido y autor (este último de forma opcional), mandando por debajo el id del comentario al que responde.

P1.

Cree un componente para renderizar comentarios y threads dentro de la plataforma, que reciba el contenido y el autor como props. En caso de que el comentario responda a otro, también debe recibir el id del comentario al que responde y mostrarlo de alguna manera. En caso contrario, no debe mostrar nada. También, si no se recibe un nombre para el autor del comentario, debe mostrar «Anónimo».

P2.

Ahora que tiene un componente para los comentarios o threads, úselo para crear la página principal donde se muestran los threads creados.

Para obtener los threads, habrá un servidor disponible con la siguiente URL: `http://fullstack.dcc.uchile.cl:3000/threads`. Deberá usar *axios* y crear un módulo para comunicarse con el servidor para obtener y crear threads. Para estas funciones es obligatorio que especifique el tipo de sus argumentos, de admitir, y de su tipo de retorno.

La respuesta del servidor tendrá la siguiente forma para threads y comentarios:

```
interface Post {  
  id: number  
  content: string  
  author: string | null  
  thread: number | null // null si es thread, un id si es comentario  
  parent: number | null // null si no responde a nadie o el id del respondido  
  createdAt: string  
  updatedAt: string  
  likes: number  
  dislikes: number  
}
```

Para crear un thread, debe hacer un POST al mismo endpoint (`http://fullstack.dcc.uchile.cl:3000/threads`). Solo se requiere que entregue el contenido y el autor, de haber. El resto es manejado por el servidor.

P3.

Extienda su modulo con funciones para obtener y crear comentarios dentro de un thread. Para ello está dispuesta la siguiente URL: `http://fullstack.dcc.uchile.cl:3000/threads/:id`, donde `:id` pertenece a algún thread. Hacer un GET a esa URL entregará la siguiente respuesta (si el id es válido):

```
interface ThreadAnswer {  
  thread: Post  
  comments: Post[]  
}
```

Para crear un comentario, debe hacer un POST a la misma URL. Acepta los mismos campos que el endpoint para threads, sumando `parent` como el id del comentario al que responde, de haber. Este id debe ser de otro comentario dentro del mismo thread, o el servidor no aceptará la petición.

P4.

Para complementar el punto anterior, ahora debe añadir la funcionalidad para acceder a la vista detallada de un thread. Para ello, debe añadir alguna estructura clickeable en el componente que renderiza una publicación (P1). Luego, modifique el componente principal de su página para mostrar la vista detallada si el usuario hace click en algún thread (puede usar **React Router**).

P5.

Añada un formulario al principio de la página principal y la vista detallada de un thread. El formulario debe pedir contenido y, opcionalmente, un autor. El formulario debe crear un thread si es que es llenado desde la página principal, pero, si el formulario es llenado desde la vista detallada de un thread, deberá, en su lugar, crear un comentario dentro del thread.

Adicionalmente, en la vista detallada de un thread, modifique el componente que usa para renderizar los comentarios de la página, para añadir un **formulario desplegable** en caso de que un usuario quiera responder a un comentario. Dicho formulario debe pedir lo mismo que el anterior, pero debe publicar un comentario respondiendo al comentario del cual fue desplegado.

P6.

Cada comentario debe mostrar su cantidad actual de likes y dislikes. El número debe actualizarse de forma reactiva al presionar el botón correspondiente, sin recargar la página.

Para poder actualizar los likes o dislikes, extienda su módulo añadiendo funciones para actualizar threads y comentarios. Puede actualizar cualquier publicación (thread o comentario) haciendo un PUT al siguiente endpoint: `http://fullstack.dcc.uchile.cl:3000/posts/:id`.

Nota: El endpoint sobrescribe el objeto. Por lo tanto, si solo quiere actualizar un campo en específico, debe entregar una copia del objeto con el campo actualizado en el cuerpo de la petición al endpoint.